

ML/AI-модуль карьерно-образовательной платформы — Agentic RAG + локальная Ollama

Этот репозиторий описывает **ML/AI часть** большой платформы карьерно-образовательных треков: модуль персонализированного подбора и семантического поиска, который можно подключать к “большому продукту” как сервис.

1) Что делает ML-часть (ценность)

- **Семантический поиск треков** по естественному языку (не по ключевым словам).
- **Персональные рекомендации** на основе профиля: навыки, специализация, описание, предпочтения.
- **Контроль и explainability**: почему выдан именно этот трек (совпавшие навыки, применённые фильтры, score).
- **Диалоговая оркестрация**: агент уточняет запрос, извлекает ограничения и ведёт stateful-сессию.

Принцип: LLM используется как “интерфейсный интеллект” (routing + формулировки), а рекомендации собираются **детерминированным пайплайном**, чтобы результат был стабильным и проверяемым.

2) Роль модуля в большой платформе

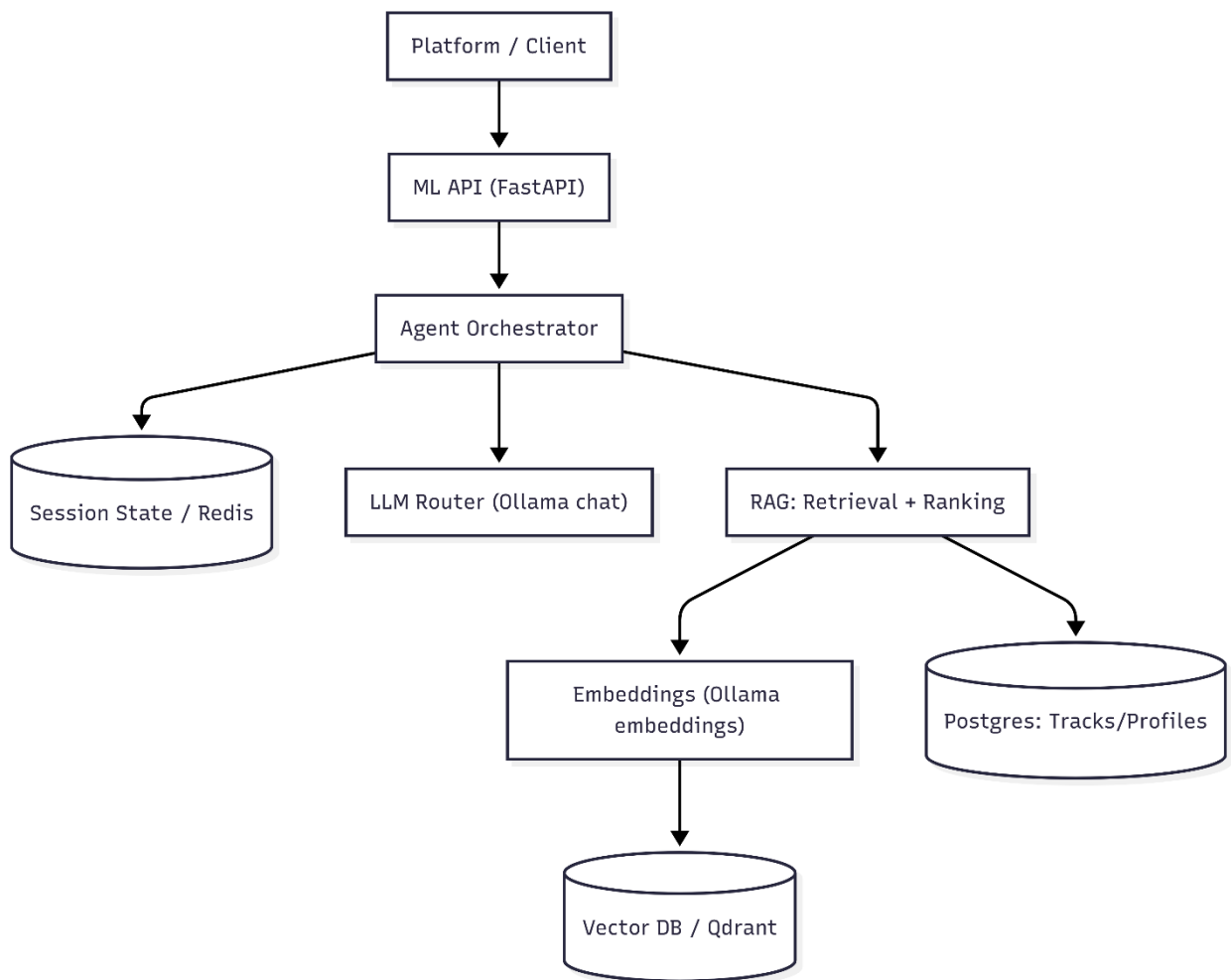
Входы из платформы:

- каталог треков (контент, требования, метаданные);
- профиль пользователя (skills/specialty/about/location/format и др.);
- событие “обновился трек/профиль” (для переиндексации/улучшения качества).

Выходы в платформу:

- список рекомендованных/найденных треков + score + объяснение;
 - состояние сессии (фильтры, последний запрос, история) для UX/аналитики.
-

3) Архитектура ML-модуля



4) Данные и сигналы (минимально необходимое)

4.1 Track (объект рекомендаций)

- `title`, `description`, `specialization`, `region`, `format`, `is_active`
- `required_skills` (веса навыков: `{skill: weight}`)
- `tasks` (список задач/результатов трека)

4.2 Profile (персонализация)

- `specialty`, `about`, `skills[]`, `location`, `employment_format`

4.3 Session State (контекст диалога)

- `filters` (направление/город/формат + extras)
- `history` (хвост диалога для self-query)
- `last` (последний инструмент + пагинация для команды “ещё”)

5) ML-пайплайн: от контента до рекомендаций

5.1 Индексация (offline/nearline)

1. Берём активные треки из Postgres.
2. Собираем “богатый текст” трека: title + description + required_skills + tasks.
3. Генерируем embedding через **Ollama embeddings**.
4. Сохраняем в Qdrant как point:
 - o `id = track.id`
 - o `vector = embedding`
 - o `payload = {specialization, region, format, is_active, ...}`

Критично: размерность вектора фиксируется моделью embeddings; при смене модели нужна переиндексация/пересоздание коллекции.

5.2 Online inference (поиск / рекомендации)

1. Пользователь пишет запрос в свободной форме.
2. Агент извлекает ограничения (format/region/specialization) и обновляет `filters` в Redis.
3. Агент выбирает инструмент:
 - o `search_tracks` (семантический поиск),
 - o `recommend_tracks` (персональный подбор),
 - o `list_tracks` (каталог по фильтрам),
 - o `get_track_by_id` (точечный запрос).
4. Retrieval:
 - o формируется `query_text` (для recommend: запрос + профиль),
 - o строится embedding запроса,
 - o Qdrant similarity search (cosine).
5. Hard-filters:
 - o `is_active`,
 - o `specialization`, `region`, `format` (матч по payload/SQL).
6. Ranking:
 - o `semantic_score` берётся из Qdrant,
 - o `skill_score` считается как доля совпавших weighted skills,
 - o итог:

```
final_score = 0.40 * semantic_score + 0.60 * skill_score
```

6. Explainability:
 - o возвращаются `matched_skills`,
 - o UI получает “карточки” с score и аргументами совпадения.

6) Какие “модели” используются сейчас (и почему это оправдано)

6.1 Embeddings

- Embedding модель: `nomiic-embed-text` (через Ollama).
- Используется для: векторизации треков и запросов.
- Плюсы для MVP: локально, дёшево по инфраструктуре, быстрый запуск, контроль данных.

6.2 LLM (agentic слой)

- Chat модель: `qwen2.5:7b` (через Ollama).
- Используется строго для:
 - self-query / routing (выбор инструмента и аргументов),
 - формулировок уточняющих вопросов/короткого follow-up.

LLM не “придумывает” рекомендации: он управляет инструментами и форматирует ответ, а ядро качества находится в retrieval+ranking.

7) Качество и метрики (как доказывать ML-ценность)

7.1 Offline evaluation (до пользователей)

- Набор тестовых запросов (по ролям: backend/frontend/data/devops/design).
- Метрики:
 - Recall@K / nDCG@K / MRR (по релевантной разметке),
 - Constraint Satisfaction Rate (доля выдач, соблюдающих region/format/specialization),
 - Skill Match Coverage (сколько “весов” навыков закрыто рекомендациями).

7.2 Online evaluation (на продукте)

- CTR по рекомендациям, “открытие карточки”, “добавить в план”, “сохранить трек”.
 - Time-to-First-Recommendation (TTFR) и латентность retrieval.
 - A/B:
 - разные формулы ранжирования,
 - разные стратегии query building (профиль+запрос vs только запрос),
 - разные embeddings/LLM модели.
-

8) MLOps/инженерия вокруг моделей (что важно для масштабирования)

- Версионирование: модель embeddings + её размерность + версия коллекции Qdrant.
- Переиндексация: при смене embedding-модели/размерности (флаг пересоздания коллекции).
- Warmup: при старте сервис прогревает Ollama (chat + embeddings) для снижения холодной задержки.
- Наблюдаемость:
 - инструмент/аргументы/фильтры/offset-limit,

- ошибки Ollama/Qdrant,
 - распределение latency (embedding, search, DB).
-

9) Интеграционный контракт (минимальный)

Префикс: `/api/v1`

- `POST /chat/stream` (SSE): выдаёт текст + теги `<TRACK_CARD ... />` для рендеринга в клиенте.
 - `GET /chat/state`: текущие filters/stage/last/history_count (для “agent trace” и отладки).
 - `POST /chat/reset`: сброс контекста сессии.
 - `GET /tracks`, `GET /tracks/{id}`: источник “объектов треков” для карточек и каталога.
-

10) Честные ограничения текущей ML-реализации

- Пока нет обучения на фидбэке (learning-to-rank / bandits) — только deterministic ranking.
 - В профиле/авторизации есть MVP-упрощения (для демо режима).
 - Фильтры по городу/формату — строковый матч; нет геопоиска/нормализации сущностей.
-

11) Roadmap именно для ML-части (что усилит moat)

- Feedback loop:
 - события кликов/сохранений → признаки → онлайн-переранжирование,
 - бандиты/контекстные бандиты для выбора стратегии выдачи.
- Hybrid retrieval:
 - смешивание dense (Qdrant) + sparse (BM25/SQL) + правила,
 - калибровка scores и стабилизация ранжирования.
- Entity normalization:
 - город/регион, формат, роли/направления (словарь + классификатор).
- Персонализация “уровня”:
 - junior/middle/senior, темп, длительность, prerequisites.